June 2026

WoxionChat

Enterprise Chatbot Platform

INTRODUCTION

Background and Rationale

- In the era of Industry 4.0, enterprises generate and manage an enormous volume of unstructured data, including contracts, standard operating procedures (SOPs), technical manuals, reports, and internal documentation.

- Sensitive corporate information may be transmitted to third-party servers, reducing organizational control over confidential data and creating long-term dependency on external providers.

INTRODUCTION

Background and Rationale

WoxionChat is an on-premise enterprise AI assistant that provides secure and intelligent access to enterprise knowledge. It leverages Agentic RAG, where autonomous agents collaboratively retrieve and reason over multiple evidence sources, and an enhanced Agentic Context Engineering (ACE) framework to continuously refine contextual knowledge, improving response quality while preserving data privacy.

INTRODUCTION

Research Objectives

The objectives of this research are as follows:

- To design and implement WoxionChat based on an Agentic RAG architecture for intelligent retrieval and reasoning over enterprise knowledge
- To improve the ACE framework by introducing more effective context management and refinement mechanisms, enabling higher-quality contextual reasoning in long-running interactions.
- To fine-tune and deploy an open-source Vietnamese large language model capable of operating entirely within private infrastructure.

THEORETICAL FRAMEWORK & METHODOLOGY

Agentic Retrieval-Augmented Generation (Agentic RAG)

Agentic RAG extends this approach with autonomous agents that plan, retrieve iteratively, reason across multiple sources, and use external tools, enabling adaptive and evidence-driven responses for complex tasks.

Context Engineering

Context Engineering is the process of selecting and organizing the most relevant information for an LLM, including retrieved knowledge, conversation history, and tool outputs. By providing concise, high-quality context, it improves reasoning accuracy, reduces irrelevant information, and enhances performance in enterprise AI applications.

Low-Rank Adaptation (LoRA)

LoRA is a parameter-efficient fine-tuning technique that freezes pretrained weights and learns lightweight low-rank updates. It significantly reduces training cost and memory usage while preserving the base model's knowledge and maintaining strong downstream performance.

THEORETICAL FRAMEWORK & METHODOLOGY

Large Language Model Fine-Tuning

- Multi-domain dataset was constructed by collecting question-answer pairs from diverse public resources, including websites, digital documents in PDF format, educational materials, and open-source datasets available on the Hugging Face platform.

- After preprocessing, deduplication, and format normalization, the initial corpus contained more than 15,000 instruction samples.

- Specifically, the Qwen3-8B was employed to evaluate each question through three independent inference attempts. The generated answers were compared with the corresponding ground-truth labels.

- After filtering, the dataset was reduced from over 15,000 samples to approximately 6,000 high-quality instruction examples suitable for reasoning-oriented training.

THEORETICAL FRAMEWORK & METHODOLOGY

Large Language Model Fine-Tuning

The final supervised fine-tuning dataset therefore consists of three components

- Instruction: the original user question or task.
- Reasoning: a detailed long-form Chain-of-Thought describing the intermediate reasoning process.
- Answer: the final ground-truth response.

Parameter	Value
Base model	Qwen3-8B
Training method	LoRA (PEFT)
LoRA rank (r)	16
LoRA alpha (α)	32
Target modules	All linear layers
Epochs	3
Per-device batch size	8
Gradient accumulation	2
Effective batch size	16
Learning rate	5×10^{-5}
Warm-up ratio	0.05
Maximum sequence length	3000 tokens
Precision	bfloat16
Attention implementation	FlashAttention-2
Gradient checkpointing	Enabled
Sequence packing	Enabled
Checkpoint interval	Every 50 steps

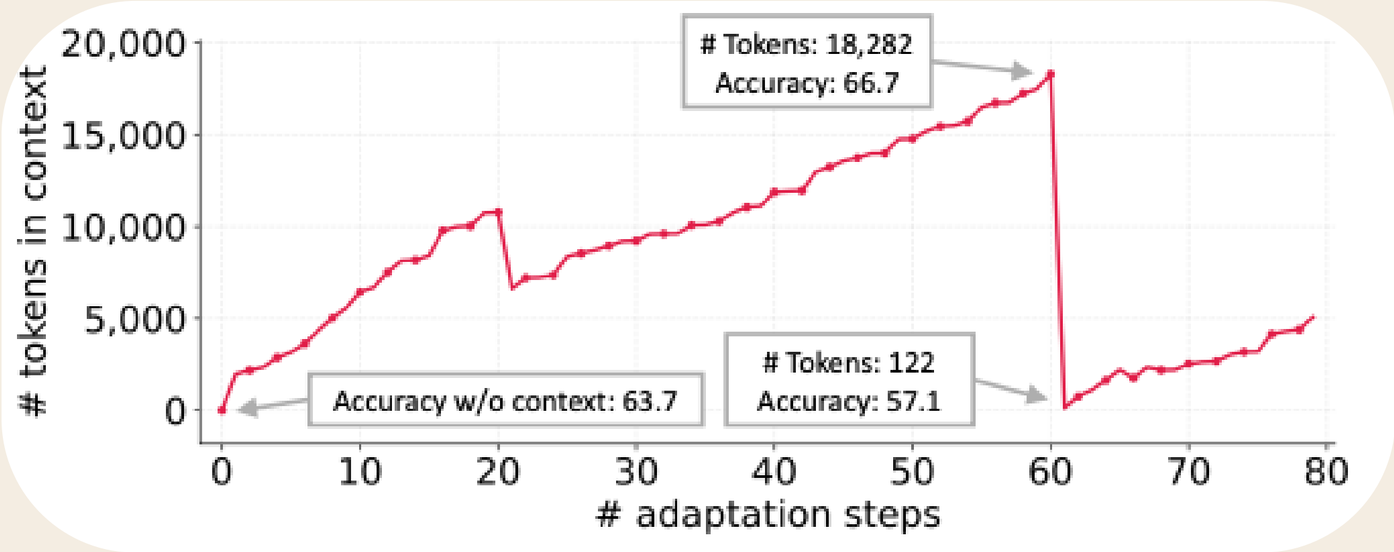
THEORETICAL FRAMEWORK & METHODOLOGY

Agentic Context Engineering (ACE)

Limitations of Context Engineering:

- *Brevity Bias:*
  - Iterative optimization tends to produce increasingly shorter contexts.
  - Important domain knowledge, reasoning patterns, and task-specific details may be lost.

- *Context Collapse:*
  - Repeated rewriting or summarization gradually discards valuable information.
  - Leads to irreversible knowledge degradation and weaker downstream performance.

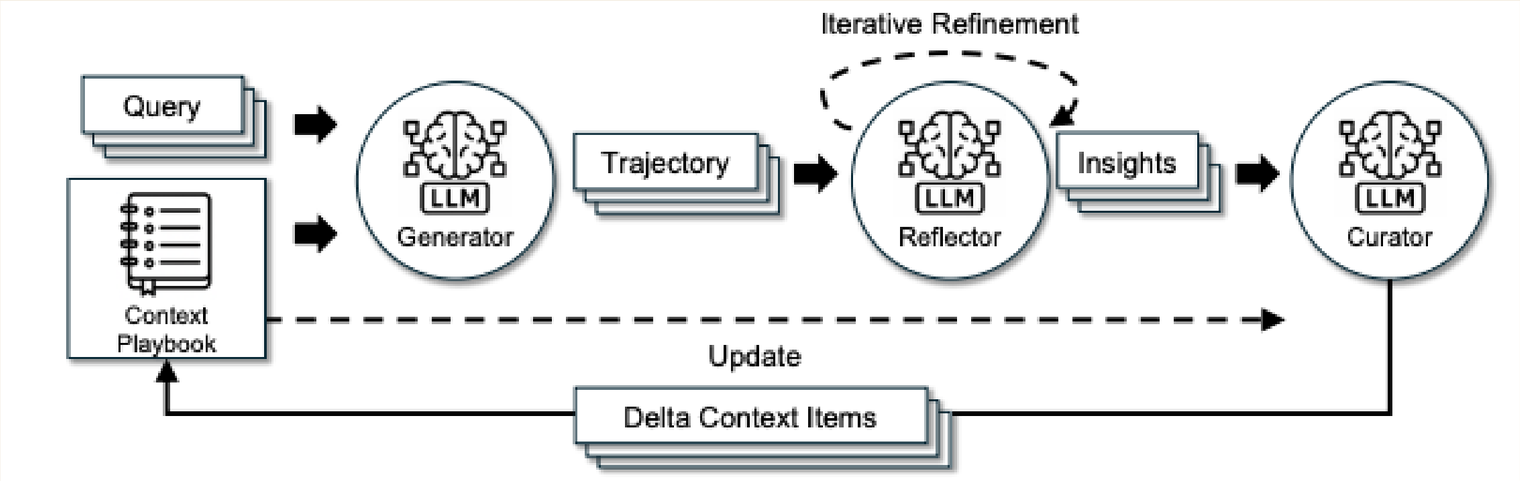


THEORETICAL FRAMEWORK & METHODOLOGY

Agentic Context Engineering (ACE)

- Agentic Context Engineering (ACE) was proposed to overcome these challenges by treating context as an evolving and structured knowledge repository rather than a monolithic prompt.

- Instead of repeatedly rewriting the entire context, ACE represents knowledge as fine-grained playbook entries that can be incrementally updated through localized modifications. New insights are appended as delta updates, existing entries are selectively refined, and redundant information is removed through a grow-and-refine mechanism.



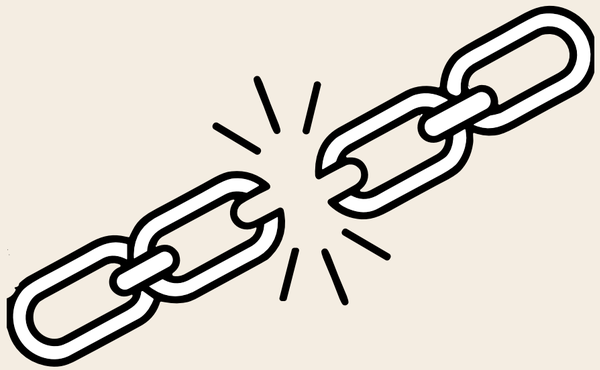
THEORETICAL FRAMEWORK & METHODOLOGY

Agentic Context Engineering (ACE)

Although ACE effectively addresses the challenges of brevity bias and context collapse. It still exhibits several limitations in practical deployment

- If the Reflector generates inaccurate analyses or hallucinates during the insight extraction process, these erroneous observations may be incorporated into the playbook by the Curator.

- The playbook continuously expands, injecting the entire collection of accumulated experiences into the model context becomes computationally inefficient and introduces substantial redundancy.

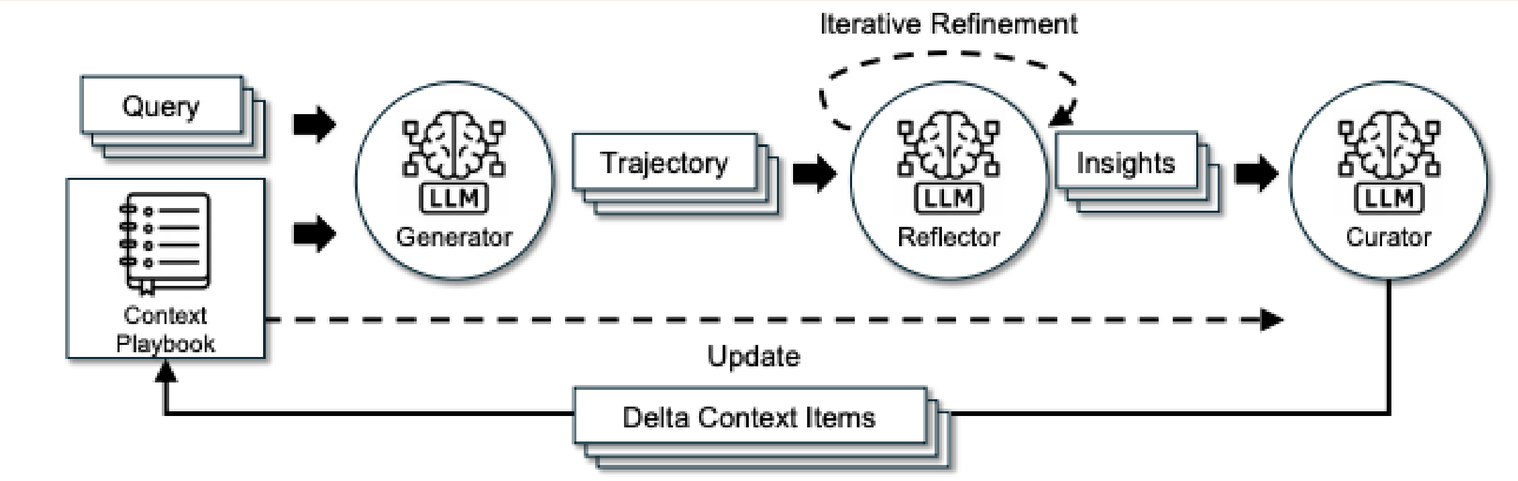


THEORETICAL FRAMEWORK & METHODOLOGY

Agentic Context Engineering (ACE)

- **Retrieval-Augmented Execution (RAE):** Instead of injecting the entire playbook into the model context, every playbook entry is encoded into a dense vector representation and indexed in a vector database. When processing a new query, semantic similarity search is performed to retrieve only the top-(K) playbook entries that are most relevant to the current task.

- **Failure Memory Bank:** Whenever the system produces an incorrect answer or exhibits suboptimal behavior, the corresponding interaction is analyzed by the Reflector, which extracts the underlying error pattern and generates corrective insights. These failure cases are then embedded and stored in a separate memory repository.



SYSTEM ARCHITECHTURE

The system consists of three major layers:

- The presentation layer provides a web-based interface through which users can upload documents, manage personal knowledge, and interact with the AI assistant.

- The application layer is responsible for user authentication, document management, request routing, and communication with the AI engine. To improve scalability and maintainability, the Agentic RAG service is deployed as an independent service

- The data layer stores user accounts, uploaded documents, vector representations, conversation memories, and self-improving knowledge bases.

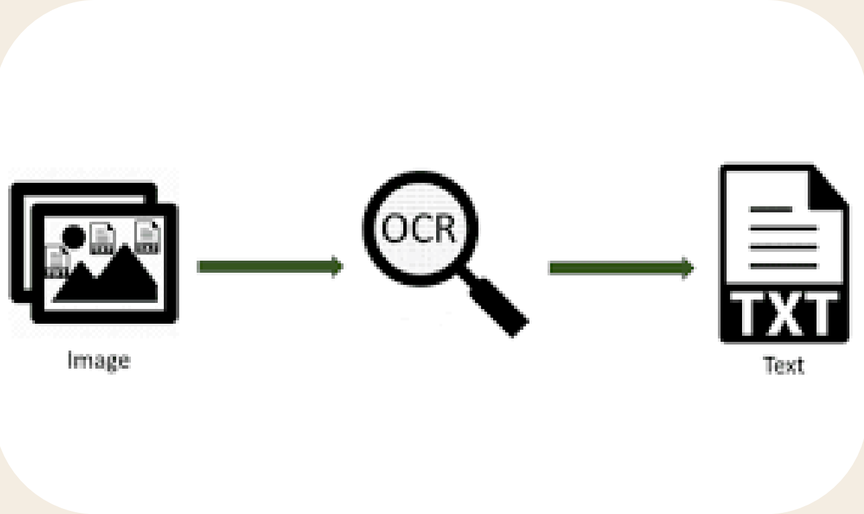
SYSTEM ARCHITECHTURE

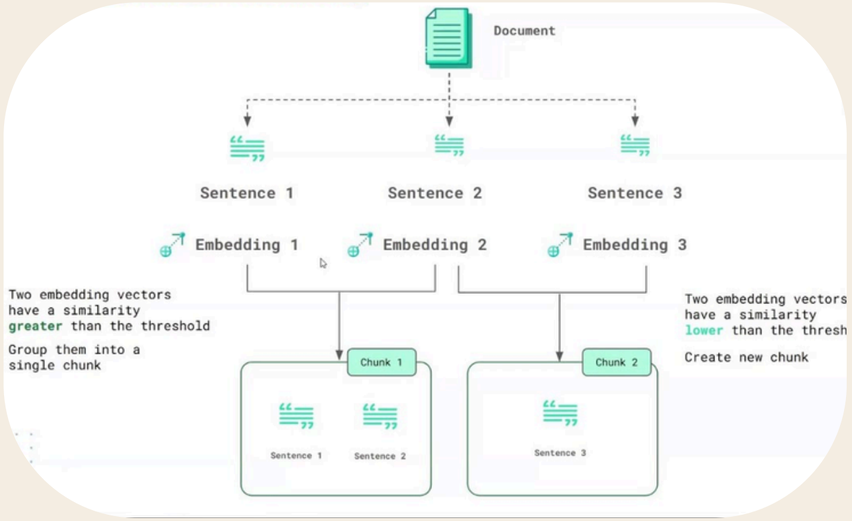
Document Upload and Management Module

Depending on the document format, OCR techniques are first applied to extract textual information from scanned files or images.

The extracted content is subsequently cleaned and divided into semantically coherent chunks rather than fixed-length segments.

To maintain data isolation, documents uploaded by administrators are incorporated into the global enterprise knowledge base, whereas documents uploaded by individual users remain accessible only within their personal workspace.





SYSTEM ARCHITECHTURE

Chatbot Interaction Module

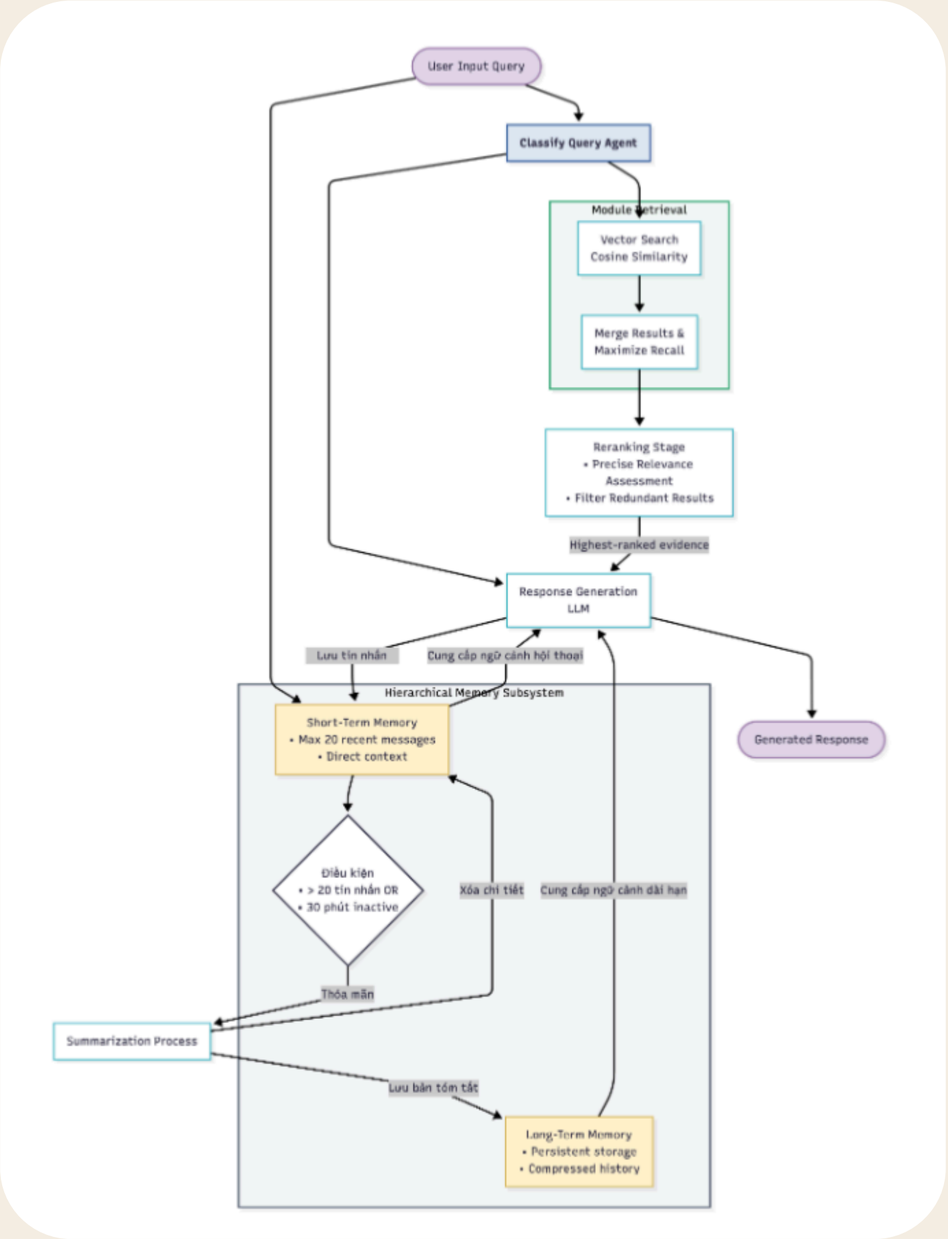
WoxionChat Agentic RAG Architecture:

- Classify Query Agent:
  - Determines whether external retrieval is needed.
  - Skips retrieval for simple queries to reduce latency and token usage.

- Vector Retrieval: Retrieves semantically relevant documents from the knowledge base.

- Reranking: Re-evaluates retrieved candidates. Selects the most relevant evidence before passing it to the LLM.

- Hierarchical Memory:
  - Short-term memory: stores the latest 20 messages for multi-turn conversations.
  - Long-term memory: summarizes older interactions into persistent knowledge to control context size.



SYSTEM ARCHITECHTURE

Self-Improving Module

- Whenever users provide corrective feedback, the system analyzes discrepancies between the generated response and the expected answer to identify useful insights.

- This selective retrieval strategy reduces unnecessary context while allowing the language model to leverage previous successful experiences and avoid repeating known mistakes. Consequently, the chatbot continuously improves its performance over time without requiring expensive model retraining.

EXPERIMENTAL RESULTS

Retrieval Benchmark

To evaluate the effectiveness of the proposed retrieval module, experiments were conducted on two Vietnamese retrieval benchmarks: **GreenNode/scifact-vn** and **GreenNode/scidocs-vn** in VN-MTEB

Method	MRR	P@1	R@1	P@3	R@3	P@5	R@5	P@10	R@10
Vector Search (Cosine)	**0.6800**	**0.6300**	**0.5795**	**0.2467**	**0.6637**	**0.1700**	**0.7223**	**0.0930**	**0.7827**
Text Search (Lexical)	0.3160	0.2500	0.2383	0.1167	0.3258	0.0880	0.3992	0.0530	0.4775
Hybrid Search (Vector + Lexical)	0.5510	0.4800	0.4412	0.2000	0.5195	0.1480	0.6270	0.0860	0.7273

Method	MRR	P@1	R@1	P@3	R@3	P@5	R@5	P@10	R@10
Vector Search (Cosine)	**0.3883**	**0.2900**	**0.0097**	**0.2167**	**0.0217**	**0.1620**	**0.0271**	**0.1100**	**0.0368**
Text Search (Lexical)	0.0786	0.0500	0.0017	0.0300	0.0030	0.0300	0.0050	0.0290	0.0097
Hybrid Search (Vector + Lexical)	0.3528	0.2500	0.0084	0.1600	0.0160	0.1460	0.0244	**0.1100**	**0.0368**

EXPERIMENTAL RESULTS

LLM Fine-tuning Evaluation

Model	STEM	Social	Humanities	Others	Average
BloomVN-8B-chat	62.81	60.47	55.40	56.56	50.72
Llama3-ViettelSolution	62.42	60.12	52.37	56.20	51.52
Vintern-3B-beta	61.01	58.41	51.98	54.81	51.70
SeaLLM-7B-v2.5	60.66	55.95	49.05	53.30	49.35
MI4uLLM-7B-Chat	58.69	56.86	52.36	52.08	44.72
Vistral-7B-Chat	57.02	55.12	48.01	50.07	43.32
DeepSeek-R1-Distill-7B	46.56	39.84	38.86	48.56	61.14
SDSRV-7B-chat	60.55	55.95	49.05	48.55	36.29
Ours (Fine-tuned Qwen3-8B)	**68.15**	**63.58**	**56.37**	**56.84**	**61.95**

EXPERIMENTAL RESULTS

Enhanced Agentic Context Engineering

Method	GT labels	Metric	Finer_0.5_offline	Formula_offline
Origin	x	Best Validation Accuracy	0.583	0.747
		Initial Test Accuracy	0.584	0.665
		Final Test Accuracy	0.513	0.700
Using RAE	x	Best Validation Accuracy	0.518	**0.787**
		Initial Test Accuracy	0.564	**0.655**
		Final Test Accuracy	0.476	**0.775**
Using Failure Memory	x	Best Validation Accuracy	**0.657**	0.740
		Initial Test Accuracy	**0.569**	0.670
		Final Test Accuracy	**0.604**	0.740
Using RAE + Using Failure Memory	x	Best Validation Accuracy	0.570	**0.787**
		Initial Test Accuracy	0.586	**0.665**
		Final Test Accuracy	0.500	**0.770**

Method	Normal_offline (%)		Challenge_offline (%)	
	Task Goal Completion	Scenario Goal Completion	Task Goal Completion	Scenario Goal Completion
Origin	20.8	8.9	6.7	0.7
Using RAE	20.2	**12.5**	8.2	0.7
Using Failure Memory	**23.2**	**12.5**	**8.9**	**2.9**
RAE + Failure Memory	21.4	10.7	7.4	2.2

THANK YOU

Thanks For Hearing